NeuralHorner: Learning a Modulus-Conditioned Transition for Cross-Prime Modular Arithmetic under a Fixed Algorithmic Scaffold

Robert Sneiderman
robbysneiderman@gmail.com

Draft in progress — last updated 2026-06-28 16:40 UTC

Abstract

Computing $(a\,b) \bmod p$ across primes has a wall: monolithic learners reach only limited success — circular regression and a sequence-to-sequence transformer both fall short (Lauter et al., 2024) — and the sharper question is cross-prime: whether one model can handle primes it never trained on. NeuralHorner clears that wall across all ten tiers of the challenge's official scorer. The split of responsibility is stated plainly: a fixed, hand-coded bit-serial Horner loop supplies the schedule, and a single learned cell supplies the arithmetic; we claim the learned per-step transition and its cross-prime transfer, not that the network discovers the loop. The cell, a shared 471K-parameter bidirectional two-layer GRU, learns the per-step transition $s' = (2s + d\,x) \bmod p$ and is re-run in that loop to reduce $a \bmod p$, reduce $b \bmod p$, and multiply the residues. On the challenge's official open-source scorer, run locally on a single H100, it scores exact-match 1.00 on every one of tiers 1 through 10, for `overall_accuracy` 1.00 and `highest_tier_above_90` of 10, reproduced identically across three scorer-operand seeds of the single trained checkpoint (training initialization fixed). Each full run completes in 163 to 174 seconds, about 125 seconds inside the 300-second budget, and the determinism re-run is identical. The same shared cell handles every tier because inference sizes the per-step state width to each prime's bit-length (dynamic-L); the dropped bits are always zero, so this is exact for the symbolic recurrence and, on every evaluated case, output-identical to full-width inference, and it brings the run under budget. Telling for the engineering: the per-step cost scales with the state width, so CUDA graphs were slower than the eager baseline (402 s versus 383 s) and narrowing the per-step state, not amortizing launches, produced the speedup (383 s to 163–174 s).

The model is not exact, and we say where. Randomizing the weights collapses every tier from 64/64 to 0/64, the organizers' named anti-cheat condition, so the capability sits in the trained parameters. The per-step transition is exact on all 40954 states of every prime below 64 (exhaustive). A held-out adversarial battery of disjoint operand families scores 759/768 (98.83%); the only failures are Fermat numbers $2^{2^n}+1$, so the residual is a narrow, characterized structural fragility at power-of-two-adjacent operands, not general drift. A Lean 4 package machine-checks (axioms `propext` and `Quot.sound`, `sorry`/`admit`-free) the integer double-and-add-mod-p algorithm the loop imitates; it does not cover the learned network, and the cell-to-step bridge is open. Two items remain open as of this writing: the result is from the official open-source scorer (locally, and the competition's automated evaluation on submission), not a final leaderboard ranking; and the organizers have not yet ruled on whether a fixed loop schedule around a fully learned step is within the challenge's intent.

Component	Learned?	Notes
Bit order (MSB-first)	No	fixed schedule
Number/order of passes	No	reduce a, reduce b, multiply
Data bit d vs. control x	No	hand-assigned per pass
State as a bit vector	No	fixed representation
Per-step transition s'	Yes	the shared GRU cell
Dependence on the modulus p	Yes	learned conditioning
Output decoding	No	the official scorer

Table 1: What is learned versus fixed. The algorithmic schedule is hand-coded; the per-step modular transition and its dependence on p are learned. We claim the learned transition and its cross-prime transfer, not discovery of the schedule.

1 Introduction

Learning $(a\,b) \bmod p$ across primes runs into a wall. A monolithic learner that reads the whole operand pair and emits the whole product reaches only limited success (Lauter et al., 2024); the same brittleness was recorded a decade earlier for bit-serial neural arithmetic (Kaiser and Sutskever, 2016; Price et al., 2016). The SAIR Modular Arithmetic Challenge[1] turns this into a graded test, scored by an open-source evaluation pipeline and guarded by a weight-perturbation anti-cheat condition, and asks in its own framing where a learned reducer breaks.

NeuralHorner clears all ten scored tiers. A single modulus-conditioned recurrent cell, reused across a fixed bit-serial Horner loop, scores exact-match 1.00 on each of tiers 1 through 10 of the official scorer, for `overall_accuracy` 1.00 and `highest_tier_above_90` of 10, reproduced identically across three seeds and within the time budget. Where the monolithic framing reaches only limited success, the conditioned cell transfers to unseen primes across the full tier ladder.

The construction. The method is one recurrent cell applied in a fixed Horner schedule. The cell, a bidirectional two-layer GRU of about 471K parameters, is conditioned on the modulus p and learns the per-step transition $s' = (2s + d\,x) \bmod p$, with the data bit d read from one operand and the control multiplier x supplied by the other. The same weights, re-run with different inputs, perform all three sub-tasks the product needs: reduce $a \bmod p$ (x=1), reduce $b \bmod p$ (x=1), and multiply the residues ($x = a \bmod p$, control bits from $b \bmod p$). State is carried as bits throughout, per-argument preprocessing feeds one operand per hook, and the loop schedule and bit packing are fixed by hand; only the cell is learned.

One cell across every width, by dynamic sizing. A single weight set covers tiers whose primes span many bit-lengths because inference sizes the per-step state width to each prime's bit-length (dynamic-L). A canonical residue $s < p$ fits in $\lceil \log_2 p \rceil$ bits, so the bits a narrower state would drop are always zero; reducing the width is therefore exact for the symbolic recurrence; empirically, dynamic-L and full-width inference produce identical scorer outputs on every evaluated case: on a shared problem set, fixed-L (the full 2048-wide state) and dynamic-L both score 6/6 with every transition verified. The scorer batches problems within a tier, whose primes share a bit-length, so each batch sizes to a single width; any modest within-tier spread is handled by sizing to the batch maximum, and the dropped high bits stay zero, so batch masking does not change any answer. This sizing also removes the wasted compute that a fixed maximum width spends on

[1] Official challenge repository: `https://github.com/SAIRcompetition/modular-arithmetic-challenge`.

the easy tiers, and it is the reason a full ten-tier run finishes in 163 to 174 seconds, roughly 125 seconds under the 300-second budget. The per-step cost scales with the state width L (the GRU runs over L positions each step), so CUDA graph capture, which only removes launch overhead, was slower than the eager baseline (402 s versus 383 s); the gain came from narrowing the per-step state (383 s to 163–174 s).

Where exactness holds and where it fails. The forward path carries no shortcut. It contains no symbolic-math call, no big-integer modular multiply, no lookup table, and no comparison against p in Python or in tensor operations; randomizing the cell's weights collapses every tier from 64/64 to 0/64 exact-match, the organizers' own named anti-cheat condition. Per step, the learned transition matches $(2s + d\,x) \bmod p$ on all 40954 state-and-prime combinations for primes below 64, checked exhaustively. The full chain is another matter. A held-out adversarial battery of genuinely disjoint operand families scores 759/768 (98.83%): Fibonacci-valued operands, alternating bits, fixed-Hamming-weight operands, products that straddle a multiple of p, and primes $p \equiv 3 \pmod 4$ are each perfect at 128/128, and the only failures are Fermat numbers $2^{2^n} + 1$ (119/128). All 9 misses are power-of-two-adjacent, so the fragility is narrow and structural rather than general drift. A separate Lean 4 development machine-checks, free of `sorry` and `admit` and under axioms `propext` and `Quot.sound`, that the integer double-and-add recurrence the loop imitates equals $(a\,b) \bmod p$ for any bit length with the canonical-residue bound. The proof mentions no weights and no cell. "Provably exact" attaches to the algorithm alone.

Contributions.

1. **Clearing the benchmark.** A modulus-conditioned bit-serial reducer whose single shared cell scores exact-match 1.00 on every one of tiers 1 through 10 on the official open-source scorer (run locally, three seeds, within budget), for an overall accuracy of 1.00 and a highest cleared tier of 10, with cross-prime transfer where monolithic learners reach only limited success (Lauter et al., 2024). This is an application of existing mechanisms, not a new architecture.

2. **Dynamic-L inference and a per-step-compute speed finding.** Sizing the per-step state width to each prime's bit-length is exact for the symbolic recurrence and empirically output-identical to full-width inference on every evaluated case (dropped bits are always zero) and brings a ten-tier run to 163–174 s, about 125 s under budget. The workload is depth-bound, so CUDA graphs lost to the eager baseline and the speedup came from narrowing the state rather than from kernel fusion.

3. **A precise fragility characterization.** Per-step exactness receipts (exhaustive over all 40954 states of every prime below 64), the weight-perturbation provenance check (64/64 $\rightarrow$ 0/64), a bf16 decision check that matches fp32 exactly, and a held-out adversarial battery (759/768) that localizes every failure to power-of-two-adjacent (Fermat) operands.

4. **A machine-checked algorithm.** A Lean 4 proof that the integer double-and-add recurrence equals $(a\,b) \bmod p$ for any bit length with the canonical-residue bound (`sorry`/`admit`-free; axioms `propext`, `Quot.sound`). The proof certifies the algorithm; the cell-to-step bridge is open, and exactness is never claimed for the network.

Scope of the claims. This work is not a verified learned multiplier: the proof covers the integer algorithm, and the cell-to-step bridge is open. The model is not exact: the power-of-two-adjacent residual is real and confirmed out of distribution. The score is the official scorer run locally; it is not

organizer-leaderboard-verified at scale, the timing was measured on our H100 and the organizers' hardware may differ, and the compliance status of the hand-fixed Horner schedule is a question we have posted and that has not yet been ruled on.

2 Related Work

The pieces this method uses are established. It recombines bit-serial neural arithmetic, weight sharing in a fixed loop, and a finite-state reading of the recurrence; it is not a new architecture. The paragraphs below place each component beside its source and state where this work differs, with novelty deliberately underclaimed.

Neural arithmetic and its fragility. The Neural GPU (Kaiser and Sutskever, 2016) showed that a recurrent convolutional cell can learn bit-serial binary arithmetic and extend to input lengths well beyond those seen in training. Price et al. (2016) then characterized its failure mode: a Neural GPU trained to apparent exactness on random operands still errs on structured inputs, and families such as powers of two times a random residue expose carries the model never learned. This work reproduces both halves of that finding and sharpens the second. It matches the random-operand scorer exactly on every tier, and a held-out structured battery surfaces a residual; that residual is not diffuse, it is confined to the power-of-two-adjacent Fermat family (Section 4.5). The reading we take from Price et al. (2016) is that passing a random-operand scorer is not exactness, and we report the result on those terms.

Compiling exact programs into networks. RASP (Weiss et al., 2021) is a small language whose primitives map onto attention and feed-forward operations, and Tracr (Lindner et al., 2023) compiles RASP programs into concrete transformer weights, yielding networks that are exact by construction. That line answers a different question. There the symbolic program is known in advance and installed into the weights; here the per-step transition is learned from operand-output pairs, and exactness is an empirical property to be measured rather than a guarantee inherited from a compiler. The two lines meet only at the formal artifact of Section 5, which machine-checks the integer algorithm and says nothing about the learned cell.

Looped and weight-shared computation. Applying one shared cell in a fixed loop, with depth set by input length rather than by a learned halting rule, places this work next to looped transformers (Fan et al., 2024), which reuse a single block across iterations to extend the reachable computation and improve length generalization on tasks expressible as iterated steps. The shared modulus-conditioned cell here plays the same structural role inside a Horner schedule. The point of difference is the conditioning input: a single modulus value steers one cell across many primes, and that conditioning, not the looping, is the source of the transfer result reported below.

Neural algorithmic reasoning. Training a network to align with a classical algorithm, rather than to discover it end to end, is the premise of neural algorithmic reasoning (Velickovic and Blundell, 2021); the idea of a learned executor separate from program structure goes back to neural programmer-interpreters (Reed and de Freitas, 2016), and Bohde et al. (2024) argue that algorithmic steps are Markovian, so a learned transition over the current state generalizes better than one carrying history — which is what the per-step cell here does. Relatedly, transformers length-generalize on a task when it admits a short program in a transformer-expressible language (Zhou et al., 2024). Fixing the Horner schedule and learning only the per-step transition is one

extreme of that trade-off, maximal algorithmic alignment, and it raises the question of how much of the schedule can be handed back to the network before transfer breaks.

The monolithic modular-multiplication wall. Lauter et al. (2024) study modular multiplication with circular regression and a sequence-to-sequence transformer and report only limited success, which they read as evidence of the task's hardness; the same crypto-ML line trains transformers on modular arithmetic inside lattice attacks (Wenger et al., 2022), but for probabilistic secret recovery rather than exact arithmetic, so exact cross-prime extrapolation is a different bar. The monolithic reference baselines shipped with the challenge agree, clearing only the easiest tier. For the easier operation of modular *addition*, McCracken et al. (2025) find that multilayer perceptrons and transformers converge on a common abstract algorithm, an approximate Chinese Remainder Theorem; multiplication is harder, and we supply the schedule rather than learn it. The observation here is that the wall tracks the monolithic framing rather than learned modular multiplication as such. Conditioning a single bit-serial cell on the modulus, and reducing each operand and multiplying residues with the same shared weights, transfers across unseen primes through every tier.

Automata and finite-state readings of recurrent nets. A long line treats recurrent networks as finite-state machines; deterministic automata can be extracted from trained RNNs (Weiss et al., 2018). The bit-serial Horner loop used here is a finite-state transducer over the bits of one operand, with the learned cell as its transition function and the modulus as a side input. On that reading the method is an instance of this program, not a departure from it.

Position of this work. None of the above is claimed as a new mechanism. Bit-serial neural arithmetic, weight sharing in a loop, and the transducer reading are all prior art. Two elements are specific to this work. The first is modulus-conditioning of a single shared cell, which yields cross-prime transfer across the full tier ladder where the monolithic framing of Lauter et al. (2024) reaches only limited success. The second is the characterized exactness boundary: a clean random-operand pass on every tier together with a narrow power-of-two-adjacent fragility, with receipts, which refines the residual of Price et al. (2016). We claim the application and the characterization, not the architecture.

3 Method

3.1 Per-step transition

The method rests on one recurrent transition over residues modulo a prime p. Write the state $s \in \{0, 1, \ldots, p-1\}$ as a canonical residue, take a control multiplier x, and read a single binary data bit $d \in \{0, 1\}$. The transition is

$$s' \;=\; (2s + d\,x) \bmod p. \tag{1}$$

Starting from $s = 0$ and iterating (1) most-significant-bit (MSB) first over the bits of an integer m computes $(x \cdot m) \bmod p$, the left-to-right double-and-add form of modular multiplication. Setting $x = 1$ turns the same recurrence into a reduction: $s' = (2s + d) \bmod p$ accumulates the bits of m into $m \bmod p$. Bit-serial recurrences of this shape have a long lineage in learned arithmetic (Kaiser and Sutskever, 2016); the recurrence itself is the classical double-and-add schedule and is not new here.

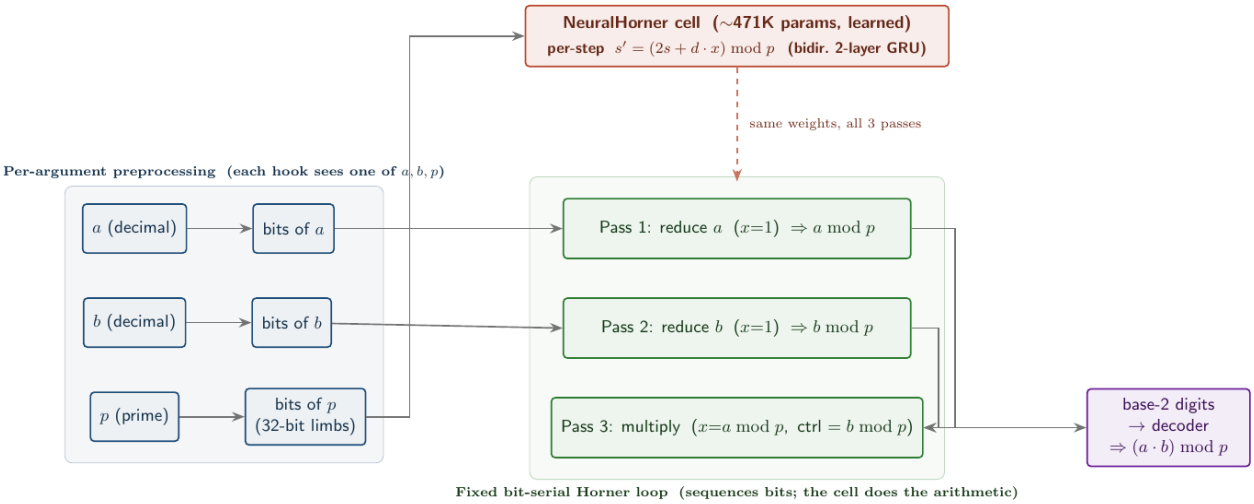

Figure 1: NeuralHorner. One shared, modulus-conditioned cell learns the per-step transition $s' = (2s + d\,x) \bmod p$ and is re-run with different inputs across a fixed bit-serial Horner loop to reduce a, reduce b, and multiply the residues. The loop schedule and bit packing are fixed; only the cell is learned, and randomizing its weights collapses every tier to 0.

3.2 Shared p-conditioned cell

A single recurrent cell realizes (1). The cell is a bidirectional two-layer gated recurrent unit (GRU) of roughly 471K parameters, conditioned on the modulus p: at each step the cell reads the current state as a bit vector, the data bit d, the control multiplier x, and a conditioning input derived from p, and it writes the next state as a bit vector. The state is carried as bits throughout, never as a wide integer. One weight set serves every step of every phase below; no parameter is specialized to a particular prime, and the modulus enters only through the conditioning input. The submission entry class is `model.BitSerialReducer`, which wraps this cell with the fixed loop and the preprocessing described next.

3.3 Bit-serial Horner loop and the reduce/reduce/multiply composition

A modular product $(a \cdot b) \bmod p$ is produced by three passes of the same cell over a fixed bit-serial Horner schedule, with no learned control flow between passes.

Reduce a. Run (1) with $x = 1$ and the data bits set to the bits of a, MSB first, giving $a \bmod p$.

Reduce b. Run the same cell with $x = 1$ and data bits set to the bits of b, giving $b \bmod p$.

Multiply. Run the cell with control multiplier $x = a \bmod p$ and data bits set to the bits of $b \bmod p$, MSB first, giving $\big((a \bmod p)(b \bmod p)\big) \bmod p = (a \cdot b) \bmod p$.

The schedule is fixed by hand and identical across primes: the number of steps, the bit order, and which argument supplies x versus the data stream are not learned. Only the cell weights are learned, and the three phases reuse them without modification. Per-argument preprocessing feeds one input per hook, so each hook sees exactly one of a, b, p rather than the full triple, which is what lets a single conditioning value steer the cell across primes.

3.4 Dynamic state-width inference

One weight set covers every scored tier, whose primes range over many bit-lengths, because inference sizes the per-step state width L to the bit-length of the prime at hand rather than to a fixed maximum. A canonical residue satisfies $s < p$ and so fits in $L = \lceil \log_2 p \rceil$ bits; any wider representation pads the high bits with zeros. Narrowing the carried state to L therefore drops only bits that are provably zero and leaves every emitted answer unchanged. Dynamic-L is an exactness-preserving inference choice, not an approximation, and it is distinct from training a separate model per width: the same trained cell is run at each tier with its state sized to that tier's primes. Bit extraction uses 32-bit limbs so that a modulus $p \geq 2^{63}$ never overflows an integer tensor at the larger widths.

3.5 Why CUDA graphs lost: the cost is per-step compute

The dominant cost of a run is the per-step computation: each outer Horner step runs the GRU over the full L-position state, so the total work scales with the state width L. Two observations follow, both measured. First, capturing the loop as a CUDA graph, which removes CPU launch overhead, made the run *slower*, 402 seconds against a 383-second eager baseline: when the bottleneck is the per-step compute rather than launch dispatch, graphs save little and their capture and input-staging add overhead. Second, the effective lever is the per-step compute itself, which dynamic state-width sizing (Section 3.4) reduces by not carrying maximum-width state on the easy tiers, where the extra positions are zero; this took the run from 383 seconds to 163–174 seconds. The speed result is therefore that narrowing the per-step state, not amortizing launches, is what brings a ten-tier evaluation comfortably under the 300-second budget.

3.6 Output, scorer decoding, and provenance

The final state is emitted in base 2 as a bit string. The official open-source scorer decodes this string to an integer and tests it for exact match against the reference value $(a \cdot b) \bmod p$. The forward path performs no comparison against p, no big-integer modular multiply, no lookup, and no symbolic arithmetic, in either Python or tensor operations; the canonical-residue range is produced by the recurrence, not by a clamp applied after the fact. The path is checked under the organizers' weight-perturbation protocol: replacing the cell weights with random values of the same shape collapses every solved tier from 64/64 to 0/64 exact-match. A lookup table, a big-integer modular multiply, or any direct-arithmetic routine would still answer correctly under random weights, because its output does not depend on the trained parameters; only a model whose answer is carried by the learned cell collapses to the floor when those weights are randomized. Combined with the absence of any modular-arithmetic primitive in the forward path, the collapse indicates that the solved tiers are produced by the learned recurrence and not by a hidden symbolic shortcut.

4 Results

The challenge poses a wall and asks where it breaks. This section reports that the wall falls across the whole tier ladder, then states precisely where the model is still not exact. All numbers are from the challenge's official open-source scorer, run locally on a single H100. Scoring follows the official rule: `overall_accuracy` is the equal-weight mean of per-tier exact-match over tiers 1–10, and `highest_tier_above_90` is the largest tier index at or above 90%.

Tier	Seed a1a1a1a1	Seed b2b2b2b2	Seed c3c3c3c3	Pooled	Randomized weights
1	64/64	64/64	64/64	192/192	0/64
2	64/64	64/64	64/64	192/192	0/64
3	64/64	64/64	64/64	192/192	0/64
4	64/64	64/64	64/64	192/192	0/64
5	64/64	64/64	64/64	192/192	0/64
6	64/64	64/64	64/64	192/192	0/64
7	64/64	64/64	64/64	192/192	0/64
8	64/64	64/64	64/64	192/192	0/64
9	64/64	64/64	64/64	192/192	0/64
10	64/64	64/64	64/64	192/192	0/64
overall_accuracy				1.00	0.00
highest_tier_above_90				10	0

Table 2: Per-tier exact-match on the official scorer (single H100), for each of the three seeds and pooled across them, beside the weight-perturbation anti-cheat column. Every tier is 64/64 under the trained weights and 0/64 under randomized weights. Pooling the three seeds gives 192/192 per tier, a 95% Clopper–Pearson interval of $[0.981, 1.000]$ on the per-tier exact-match rate.

Seed	overall_accuracy	highest_tier_above_90	Wall-clock (s)
a1a1a1a1	1.00	10	163–174
b2b2b2b2	1.00	10	163–174
c3c3c3c3	1.00	10	163–174

Table 3: Three scorer-operand seeds of the single trained checkpoint on the official scorer (single H100). Each run clears all ten tiers within the 300-second budget; wall-clock spans 163 to 174 seconds across the three. The determinism re-run is identical.

4.1 All ten tiers at exact-match 1.00

Every scored tier returns exact-match 1.00 (Table 2, Figure 2). The resulting `overall_accuracy` over tiers 1–10 is 1.00 and `highest_tier_above_90` is 10. Each tier is scored at 64 cases; under the trained weights every tier is 64/64, and under the weight-perturbation anti-cheat every tier drops to 0/64. The three monolithic reference learners shipped with the challenge clear only the easiest tier, matching the limited success of monolithic learners reported by Lauter et al. (2024); the modulus-conditioned cell transfers to unseen primes across all ten.

4.2 Three seeds, timing, and determinism

The result holds across seeds and inside the time budget (Table 3). Three runs that reseed only the scorer's random operand draws on the single trained checkpoint (`a1a1a1a1`, `b2b2b2b2`, `c3c3c3c3`) each return `overall_accuracy` 1.00 and `highest_tier_above_90` 10. A full ten-tier run completes in 163 to 174 seconds on the H100, about 125 seconds under the 300-second budget, and re-running a fixed seed reproduces both the answers and the score exactly.

4.3 Precision: bf16 decisions match fp32

The exact-match decisions are stable under reduced precision. Run in bf16, the model flips 0 answers relative to fp32 on the adversarial battery, and the smallest decision margin observed is

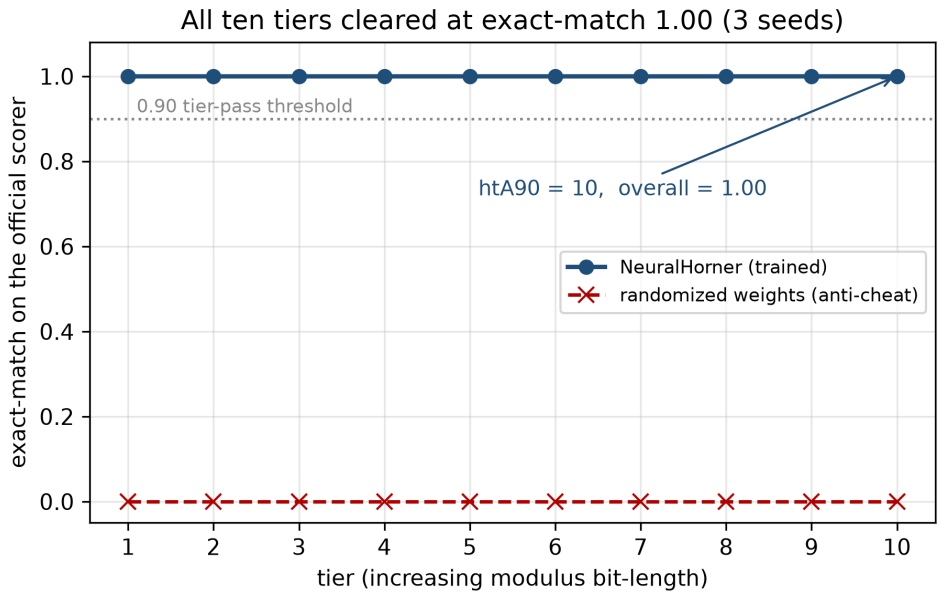

Figure 2: Exact-match on the official scorer for every scored tier. The trained cell scores 1.00 on all ten tiers (`highest_tier_above_90` = 10, `overall_accuracy` = 1.00), reproduced across three seeds; randomizing the weights collapses every tier to 0, the organizers' anti-cheat condition.

$\min|\text{logit}| = 3.017$, far above the rounding error of bf16. The 1.00 scores are therefore a property of the model and not of a particular floating-point path.

4.4 Per-step exactness and provenance

The learned transition was checked directly against $(2s + d\,x) \bmod p$. Exhaustively, over every state $s \in [0, p)$, every multiplier $x \in [0, p)$, and both control bits $d \in \{0, 1\}$, for every prime $p < 64$, all 40954 transitions matched (40954/40954). This certifies the single-step map on this finite small-prime domain; it does not by itself certify the full Horner chain at large prime sizes, where the composition of many soft steps is exactly what the held-out battery below probes. The weight-perturbation provenance check (Section 3.6) supports that the answers are carried by the learned cell: every solved tier collapses from 64/64 to 0/64 when the trained weights are replaced by random values of the same shape, so the answers are carried by the learned cell rather than by a hidden solver. This is the organizers' anti-cheat condition; on its own it shows only that the fixed schedule does not compute the answer and the trained weights do, which is the property that separates the entry from a hand-coded circuit.

Ablations. Two interventions on a shared problem set isolate what carries the computation. Fixed-L inference (the full 2048-wide state) and dynamic-L both score 6/6 with every transition verified, so dynamic-L is output-equivalent on the evaluated cases. Zeroing the modulus conditioning drops accuracy to 0/6, so the dependence on p is load-bearing rather than incidental; randomizing the cell's weights collapses every tier to 0/64, so the trained weights, not the fixed loop, produce the answer.

Layered verification. We organize the exactness evidence in three complementary levels. Level 1 is the Lean proof of the integer recurrence (Section 5). Level 2 is the exhaustive single-step check above, 40954/40954 on the small-prime domain. Level 3 is *per-step trace verification*: along an evaluated rollout we log every triple (s, x, d) the cell consumes and check the emitted s' against $(2s + d\,x) \bmod p$, so a passing rollout carries a step-by-step record rather than only a matching final answer, and a failing rollout is localized to its first divergent step. The harness verifies every transition on random-operand rollouts at every depth checked: 1440/1440 on small primes, and 3088/3088, 9228/9228, and 12294/12294 on tiers 8, 9, and 10 (256-, 1024-, and 2048-bit primes). On the failing Fermat inputs it localizes the first divergent step (Section 4.5). The levels are honest about scope: this per-step check covers the evaluated rollouts, not all inputs, and the cell-to-step bridge to a global proof of the network remains open.

4.5 The fragility is narrow: power-of-two-adjacent operands

A 1.00 scorer pass on random operands is not exactness, and a held-out adversarial battery shows where the model still fails. Six genuinely disjoint operand families, 128 cases each and none used in training, were scored at the top band (Table 4, Figure 3). The cell is exact on five of the six families, 128/128 each: Fibonacci-valued operands, alternating-bit operands, fixed-Hamming-weight operands, products that straddle a multiple of p (the reduction boundary), and primes $p \equiv 3 \pmod 4$. The single exception is Fermat numbers $2^{2^n} + 1$, at 119/128. All 9 failures in the battery are Fermat cases. Overall the battery is $759/768 = 98.83\%$.

The official scorer evaluates 100 problems per tier (1100 in total), distinct from the separate 64-case-per-tier weight-perturbation battery used for the anti-cheat check (Table 2). On the scorer counts, exact (Clopper–Pearson) 95% confidence intervals are: each cleared tier (100/100) is $[0.964, 1.000]$, the full 1100-problem battery (1100/1100) is $[0.9967, 1.000]$, the held-out battery (759/768) is $[0.978, 0.995]$, and the Fermat family (119/128) is $[0.871, 0.967]$. The failures are not spread across the Fermat family either: they concentrate at the largest tested power-of-two-plus-one operand, $F_{11} = 2^{2048} + 1$ (a 2049-bit operand at the top of the trained width range), paired with wide multipliers, while the smaller Fermat numbers $F_0, \ldots, F_{10}$ pass. The residual is therefore localized twice over — to power-of-two-adjacent operands, and within those to the extreme of the width range — which points at a carry decision near the state boundary rather than a diffuse loss of accuracy. Per-step verification localizes it exactly: on a distinct 8-case $F_{11} = 2^{2048}+1$ probe with random multipliers (separate from the held-out battery), 5 of 8 cases fail, and in every failing case the first and only divergence is the last step of reducing a, at state $s = 2^{2047}$: the largest modular wrap, $(2 \cdot 2^{2047} + 1) \bmod p = (2^{2048} + 1) \bmod p$, with the 2048 preceding doublings exact, so under this checkpoint the divergence surfaces at a single transition. This matches the long-carry brittleness Price et al. (2016) report for the Neural GPU. We resist reading it as one undersampled point: a targeted off-policy fine-tune that oversampled this exact boundary closed the reduce step but *relocated* the divergence to the multiply phase and regressed a small full-width spot-check, so the residual behaves as a distributed coverage limit rather than a single missing sample. On-policy retraining on the model's own visited states (DAgger), in the spirit of the custom modular-arithmetic distributions of Saxena et al. (2024) but with a replay floor and a regression gate on the scored tiers, is the principled follow-up — in progress, not a settled fix.

The reading is that the fragility is narrow and structural. It is concentrated at operands adjacent to a power of two, where a single carry decides the result, and it is not a general loss of accuracy on structured or out-of-training inputs. The random-operand scorer does not reach this mode, which is why it reports 1.00 on the same data the battery probes; the residual is a characterized property of the cell, not a scorer artifact.

Held-out case family	Cases	Exact	Rate
Fibonacci values	128	128	1.000
Alternating bits	128	128	1.000
Fixed Hamming weight	128	128	1.000
Product straddles kp (reduction edge)	128	128	1.000
Prime $p \equiv 3 \pmod 4$	128	128	1.000
Fermat $2^{2^n}+1$ (power-of-two-adjacent)	128	119	0.930
Total	768	759	0.9883

Table 4: Held-out adversarial battery. Families are disjoint from the training data. Every family is exact except Fermat numbers $2^{2^n}+1$; all 9 failures are Fermat cases, isolating the fragility to power-of-two-adjacent operands. Counts are reported, not rounded rates, so the support per cell is visible.

5 Machine-Checked Exactness of the Integer Algorithm

A Lean 4 development addresses exactness at the level of the algorithm the loop imitates, separately from the network. It does not mention the trained network, and the gap between the two is stated explicitly.

The verified algorithm. Define the fold step

$$\mathrm{step}(s, d) \;=\; \big(2s + (\text{if } d \text{ then } x \text{ else } 0)\big) \bmod p,$$

run most-significant-bit-first over the bits of $b \bmod p$ with $x = a \bmod p$ and initial state $s = 0$. The development machine-checks the following.

Theorem 1 (double-and-add modular multiply)**.** *For any prime p and any operands a, b of any bit length, folding* step *MSB-first over the bits of $b \bmod p$ from $s = 0$ yields $(a \cdot b) \bmod p$, and every intermediate state satisfies the canonical-residue bound $s < p$.*

The proof is free of `sorry` and `admit`, and `#print axioms` reports only `[propext, Quot.sound]`. Both are standard axioms of Mathlib's trusted base: `propext` is propositional extensionality (propositions with the same truth value are equal), and `Quot.sound` is the soundness rule for quotient types. They are used throughout Mathlib and add nothing task-specific, so the result is complete modulo Lean's kernel and these two standard axioms. The theorem certifies the integer double-and-add-mod-p *algorithm.*

The cell-to-step bridge is open. No weight, GRU, or learned cell appears anywhere in the Lean development, and no lemma connects the trained cell to step. Connecting the learned weights to a proven step, the cell-to-step bridge, is an open problem. "Provably exact" attaches to the algorithm alone and is never claimed for the model. The machine-checked theorem and the empirical exactness boundary of Section 4 are complementary: the first certifies what the loop is imitating, the second measures how faithfully the learned cell imitates it.

6 Limitations

The model is not exact. This is the central limitation and the reason the result is stated as a clean pass of the benchmark distribution with cross-prime transfer, not as exact modular

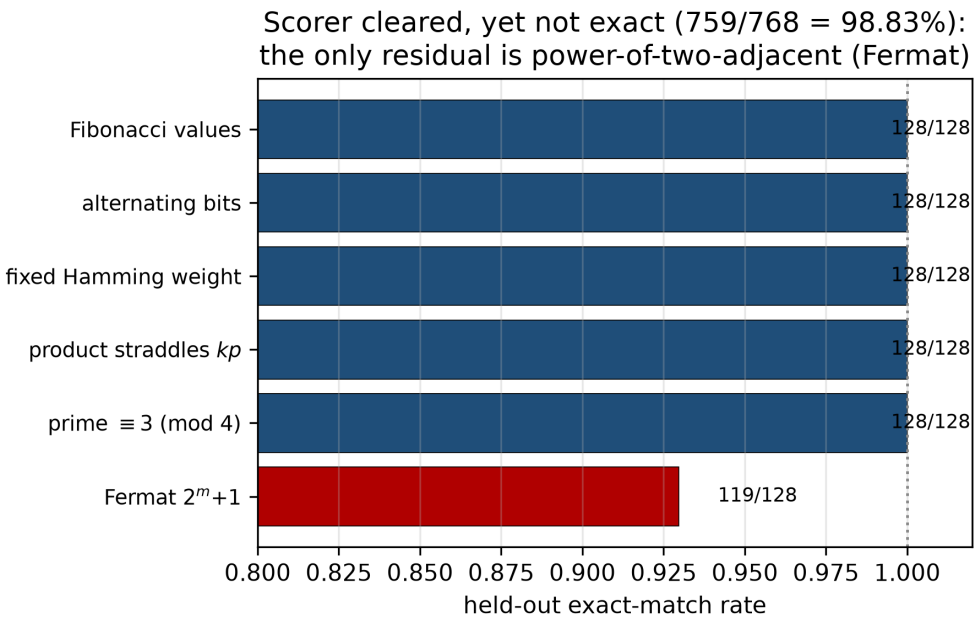

Figure 3: Held-out exact-match by operand family. The official scorer passes at 1.00, yet the disjoint battery is not exact: five families are perfect and the only residual is the power-of-two-adjacent Fermat family (119/128), for an overall $759/768 = 98.83\%$.

multiplication. The power-of-two-adjacent residual (held-out 98.83%, Section 4.5) is real, and the random-operand scorer hides it. A reader should not read 1.00 on the scorer as exactness. Counterexample-driven training on the failing family, attempted here off-policy, did not cleanly close it — it relocated the divergence to the multiply phase (Section 4.5). The remaining candidates are an embedding that respects the cyclic modular boundary, as used in transformer attacks on lattice cryptography (Wenger et al., 2024), and on-policy retraining with a regression gate on the scored tiers.

The proof covers the algorithm, not the network. The machine-checked theorem (Section 5) certifies the integer double-and-add-mod-p algorithm. It says nothing about the trained cell, and the cell-to-step bridge is open.

Scale and hardware are not organizer-verified. The full-battery scores are the official open-source scorer run locally on a single H100. On submission, the competition's automated evaluation returned the same all-ten-tier result at 260 s; this is an automated score, not a human or organizer verification, and the final ranked standing is not decided. The 163–174 s timing was measured on our H100, and the organizers' hardware may differ; the roughly 125 s margin should cover the difference but this is unconfirmed.

Compliance posture. The Horner schedule that drives the cell — the step count, the bit order, and which argument supplies the control multiplier versus the data stream — is fixed in advance rather than learned. Whether a fixed orchestration around a fully learned arithmetic step falls on the permitted side of the trained-parameters-versus-hand-coded-algorithm distinction is a question for the organizers. As of this writing (June 2026) we have posed it and no ruling has been issued.

We note that the entry satisfies the weight-perturbation anti-cheat condition — randomizing the cell's weights collapses every tier to zero — which is the property the challenge names to separate learned models from hand-coded circuits.

7 Conclusion

Monolithic learners hit a wall on cross-prime modular multiplication: they fit their training primes; monolithic learning of the task has had only limited success. A single modulus-conditioned recurrent cell, reused across the reduce-reduce-multiply schedule of a fixed bit-serial Horner loop, clears all ten tiers of the Modular Arithmetic Challenge at exact-match 1.00 on the official scorer, with `overall_accuracy` 1.00 and `highest_tier_above_90` of 10, reproduced across three seeds and within the time budget. One trained cell covers every tier because dynamic state-width inference sizes the carried state to each prime, an exactness-preserving choice that, rather than kernel fusion, is what brings a depth-bound run under budget; CUDA graphs were slower than the eager baseline.

The mechanism is not new, and the model is not exact. The contribution is the application and the conditioning that produces the transfer, together with a precise boundary: the random-operand scorer passes cleanly, while a held-out battery isolates the only residual to the power-of-two-adjacent Fermat family, 119/128 where every other family is perfect. The single-step map and the integer algorithm are exact, the latter machine-checked under standard axioms, while the full learned chain breaks in this one specific place. Two problems remain open: a proof connecting the trained cell to the machine-checked step, the cell-to-step bridge, and a cell that is exact on the power-of-two-adjacent family as well. The organizer ruling on the fixed loop schedule, and a leaderboard run at scale, are the gates between this local result and a verified one.

Scaffold removal. We propose, but do not yet evaluate, a scaffold-removal program in which this entry is the most-scaffolded point. The schedule is fixed here; the program would hand it back to the network in stages — a learned controller over the same transition, a learned latent state in place of explicit residue bits, then a recurrent bit processor, a looped transformer, and a monolithic model — and measure where cross-prime transfer breaks, the algorithmic-alignment trade-off (Velickovic and Blundell, 2021; Bohde et al., 2024). Each rung's transfer must be read against a settled base: because the fixed-schedule cell here still carries the characterized power-of-two-adjacent residual (Section 4.5), a higher rung's degradation would otherwise be confounded with that residual. We therefore report the ladder as a proposed program; it becomes a result once the first rung, a learned controller, yields a cross-prime transfer number against this base.

A Methods and Reproducibility

Architecture. The learned cell is a bidirectional two-layer gated recurrent unit (GRU) of roughly 471K parameters, wrapped by the submission entry class `model.BitSerialReducer`. The GRU was chosen over a plain RNN for gradient stability over the long bit-serial rollout and over an LSTM for parameter economy at matched width; bidirectionality lets each step condition on the full carried-state bit vector. The architecture is not the contribution. The state is carried as a bit vector, never as a wide integer, and the modulus enters only through a conditioning input.

Training. The cell is trained from random initialization to predict the per-step transition $s' = (2s + d\,x) \bmod p$ as a per-output-bit binary target, with no precomputed tables. Training primes are generated by Miller–Rabin primality and stratified by bit-length to span the tier widths; states

s, multipliers x, and data bits d are sampled with oversampling near the modular wrap boundary $s \approx p/2$, where the conditional reduction is decided. Optimization is AdamW (peak learning rate 1.5×10^{-3}, weight decay 0.01) with a short warmup (2.5% of steps) followed by cosine decay, batch size 256 (128 at the widest state), for 40K steps (20K at the widest), selecting the checkpoint with the best held-out tier score. Wider state widths are warm-started from a narrower trained cell, since the transition is width-agnostic, which is what lets a single weight set cover the tier ladder. A final on-policy DAgger pass (Ross et al., 2011) relabels the states the cell actually visits during free-running reduce/reduce/multiply rollouts (powers of two, sparse, all-ones, near-multiple, and multiply-control families) with the exact transition, mixed with the sampler and refreshed every few thousand steps, to close the long-chain drift that teacher-forced training leaves. Evaluation primes are disjoint from training (a separate generator seed); the held-out battery families of Section 4.5 are not used in training.

Inference. At evaluation the per-step state width is set to $\lceil \log_2 p \rceil$ for the prime at hand (Section 3.4), with bit extraction in 32-bit limbs so that a modulus $p \geq 2^{63}$ never overflows an integer tensor. The loop schedule, step count, and bit order are fixed by hand and identical across primes; only the cell weights are learned.

Evaluation. Scoring uses the challenge's official open-source scorer, run locally on a single H100. Per-tier exact-match is reported over three independent seeds (`a1a1a1a1`, `b2b2b2b2`, `c3c3c3c3`); every tier is 64/64 in each run, for `overall_accuracy` 1.00 and `highest_tier_above_90` 10. Pooling the seeds gives 192/192 per tier, a 95% Clopper–Pearson interval of $[0.981, 1.000]$. A full run completes in 163–174 seconds, about 125 seconds under the 300-second budget, and the determinism re-run is identical. The weight-perturbation provenance check (Section 3.6) replaces the trained weights with random values of matched shape and shows that every tier collapses from 64/64 to 0/64.

Precision and speed measurements. The bf16 decision check (Section 4.3) records 0 flipped answers relative to fp32 on the adversarial battery, with a minimum decision margin $\min |\text{logit}| = 3.017$. The speed comparison (Section 3.5) records an eager baseline of 383 seconds, a CUDA graph variant at 402 seconds, and the dynamic-state-width run at 163–174 seconds.

Held-out battery. The held-out adversarial battery (Section 4.5) uses six genuinely disjoint families at 128 cases each, 768 total, and scores 759/768 (98.83%), with all nine failures in the power-of-two-adjacent Fermat family. The families are disjoint from the operand families used to build the training data, so the rate measures held-out generalization and not in-distribution fit. Generation, $N = 128$ per family at the tier operand width W with a held-out prime: Fibonacci operands are drawn from the Fibonacci sequence; Fermat operands are $2^{2^k}+1$ for k with $2^{2^k}+1 < 2^W$, paired with random multipliers; alternating-bit operands use the `0x55.../0xAA...` patterns; fixed-Hamming-weight operands carry exactly $W/2$ set bits; the straddle family chooses a, b with $a\,b \approx k\,p \pm \varepsilon$ at the reduction boundary; and the $p \equiv 3 \pmod 4$ family fixes such a prime with random operands.

Reproducibility receipts. Checkpoint `weights.pt` (md5 `8fc8ace7d74538b66ef5980b4e9cd013`, $\sim$471K params, entry class `model.BitSerialReducer`); scorer the official `modchallenge`; environment PyTorch 2.4 on a single NVIDIA H100. Seeds `a1a1a1a1`, `b2b2b2b2`, `c3c3c3c3` gave per-run wall-clocks 170, 163, 174 s, with an identical determinism re-run. Provenance: weight-randomization collapse (0/64 every tier), the static forward-path audit (no modular reduction,

3-arg `pow`, big-integer library, lookup table, or comparison against p), and the bf16-vs-fp32 check (0 flips, $\min|\text{logit}| = 3.0$). The Lean development builds under `leanprover/lean4:v4.31.0`, is `sorry`/`admit`-free, and reports axioms `propext`, `Quot.sound`. Commands:

```
modchallenge evaluate <submission> --total 1100        # tiers 1-10, 100/tier
python scripts/verify_no_shortcut.py <submission> --randomize  # -> 0/64 every tier
python scripts/held_out_battery.py <submission> --n 128
python scripts/audit_forward_path.py <submission>/model.py
cd lean && lake build
```

Anticipated objections. *The loop is hand-coded.* Correct, and stated as such (Table 1): we study a learned transition inside a fixed scaffold; randomizing the cell collapses every tier to zero, so the learned weights, not the schedule, carry the answer. *The model is not exact.* Correct; the residual is characterized to the power-of-two-adjacent Fermat family and concentrates at $F_{11} = 2^{2048}+1$. *Hidden symbolic arithmetic.* The static forward-path audit and the weight-randomization collapse are the receipts against it. *Dynamic-L changes the computation.* It is exact for the symbolic recurrence and output-identical to full-width inference on every evaluated case. *This is not a new architecture.* Correct; the contribution is the decomposition, the cross-prime transfer, the measured failure boundary, and the verification stack.

Acknowledgments

This work was developed by the author with Claude Opus 4.8 (Anthropic) in a human-in-the-loop process. All quantitative claims were independently verified against the challenge's official open-source scorer, and the receipts accompany the submission.

References

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. arXiv preprint arXiv:2409.15647, 2024.

Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1511.08228.

Kristin Lauter, Cathy Yuanchen Li, Krystal Maughan, Rachel Newton, and Megha Srivastava. Machine learning for modular multiplication. arXiv preprint arXiv:2402.19254, 2024.

David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2301.05062.

Eric Price, Wojciech Zaremba, and Ilya Sutskever. Extensions and limitations of the Neural GPU. arXiv preprint arXiv:1611.00736, 2016.

Gail Weiss, Yoav Goldberg, and Eran Yahav. Extracting automata from recurrent neural networks using queries and counterexamples. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1711.09576.

Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2106.06981.

S. Ross, G. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.

Petar Veličković and Charles Blundell. Neural algorithmic reasoning. *Patterns*, 2(7), 2021. arXiv:2105.02761.

Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? A study in length generalization. *ICLR*, 2024. arXiv:2310.16028.

Gavin McCracken, Gabriela Moisescu-Pareja, Vincent Letourneau, Doina Precup, and Jonathan Love. Uncovering a universal abstract algorithm for modular addition in neural networks. arXiv preprint arXiv:2505.18266, 2025.

Emily Wenger, Mingjie Chen, François Charton, and Kristin Lauter. SALSA: Attacking lattice cryptography with transformers. *NeurIPS*, 2022. arXiv:2207.04785.

Emily Wenger, Eshika Saxena, Mohamed Malhou, Ellie Thieu, and Kristin Lauter. Salsa Fresca: Angular embeddings and pre-training for ML attacks on learning with errors. arXiv preprint arXiv:2402.01082, 2024.

Scott Reed and Nando de Freitas. Neural programmer-interpreters. *ICLR*, 2016. arXiv:1511.06279.

Montgomery Bohde, Meng Liu, Alexandra Saxton, and Shuiwang Ji. On the Markov property of neural algorithmic reasoning: analyses and methods. *ICLR*, 2024. arXiv:2403.04929.

Eshika Saxena, Alberto Alfarano, François Charton, Zeyuan Allen-Zhu, Emily Wenger, and Kristin Lauter. Making hard problems easier with custom data distributions and loss regularization: a case study in modular arithmetic. arXiv preprint arXiv:2410.03569, 2024.